Handwritten Digit Classification Using Neural Networks on the MNIST Dataset

A Project Report

Submitted in partial fulfilment of the requirements for the course

ARTIFICIAL INTELLIGENCE

Submitted by

DEEPESH TAHLANI

Under the Guidance of

Mr. MD Ameen

Project Guide

Department of Artificial Intelligence

Naviotech Solution Pvt. Ltd.

Project Duration: September - October 2026

Academic Year 2026 - 2027

CERTIFICATE

This is to certify that the project report entitled

"Handwritten Digit Classification Using Neural Networks on the MNIST Dataset"

is a bonafide record of the project work carried out by Deepesh Tahlani , a student of the Artificial Intelligence course, Department of Artificial Intelligence, Naviotech Solution Pvt. Ltd., under the guidance of Mr. MD Ameen, during the period September 2026 to October 2026, in partial fulfilment of the requirements of the Academic Year 2026-2027.

The project work presented in this report is the outcome of the student's own efforts, and the results described herein have been obtained from the actual implementation of the handwritten digit classification system developed as part of this project.

Project Guide

Mr. MD Ameen

Place: Naviotech Solution Pvt. Ltd.

Date: October 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who supported and guided me throughout the successful completion of this project on handwritten digit classification using neural networks on the MNIST dataset.

First and foremost, I am deeply thankful to my project guide, Mr. MD Ameen, for his valuable guidance, continuous encouragement, and constructive suggestions at every stage of this project. His expertise in artificial intelligence and machine learning helped me understand the theoretical foundations of neural networks and apply them to a practical image classification problem.

I extend my sincere thanks to the faculty members and staff of the Department of Artificial Intelligence, Naviotech Solution Pvt. Ltd., for providing the learning environment, computing resources, and academic support necessary for carrying out this work.

I am also grateful to the developers and maintainers of the open-source tools used in this project, including Python, TensorFlow, Keras, NumPy, OpenCV, and Matplotlib, and to the Google Colab platform, which provided free access to cloud-based computing resources for training the neural network model.

Finally, I thank my family and friends for their constant motivation and support during the course of this project.

Deepesh Tahlani

ABSTRACT

Handwritten digit recognition is one of the most widely studied problems in the fields of machine learning and computer vision. It forms the technological basis of numerous real-world applications, including postal code sorting, bank cheque processing, form digitisation, and automatic data entry. The objective of this project is to design, implement, and evaluate a handwritten digit classification system based on an artificial neural network trained on the MNIST dataset.

The MNIST dataset is a benchmark collection of 70,000 grayscale images of handwritten digits, each of size 28 by 28 pixels, divided into 60,000 training images and 10,000 testing images, with ten output classes corresponding to the digits 0 to 9. In this project, a feed-forward artificial neural network was implemented using TensorFlow and Keras in the Python programming language on the Google Colab platform. The pixel intensities of the images were normalised to the range 0 to 1 before training, and the network was trained for 10 epochs using the Adam optimisation algorithm with the sparse categorical cross-entropy loss function, reserving 10 percent of the training data for validation.

The experimental results show that the model achieved a final training accuracy of 98.93 percent with a training loss of 0.0342, and a final validation accuracy of 97.40 percent with a validation loss of 0.1222 after ten epochs of training. The training accuracy improved consistently across epochs, while the validation accuracy stabilised near 97 percent, indicating good generalisation with only mild overfitting in the later epochs.

In addition to model training, a predictive system was implemented using the OpenCV library, which allows a user to supply a custom handwritten digit image, preprocesses the image through grayscale conversion, resizing, normalisation, and reshaping, and predicts the corresponding digit using the trained model. The system successfully recognised a sample handwritten digit 3 provided by the user. The results demonstrate that a relatively simple neural network can deliver highly accurate handwritten digit recognition and can serve as a foundation for more advanced document analysis systems.

Keywords: Artificial Intelligence, Machine Learning, Neural Networks, Deep Learning, MNIST, Handwritten Digit Recognition, TensorFlow, Keras, OpenCV, Image Classification.

TABLE OF CONTENTS

(Right-click the table of contents and choose Update Field to refresh page numbers.)

TOC \o "1-3" \h \z \u

Certificate....ii

Acknowledgement....iii

Abstract....iv

List of Figures....vi

List of Tables....vii

List of Abbreviations....viii

Chapter 1: Introduction....1

Chapter 2: Literature Review....5

Chapter 3: Methodology....9

Chapter 4: System Design....14

Chapter 5: Implementation....18

Chapter 6: Results and Discussion....24

Chapter 7: Conclusion....29

Chapter 8: Future Scope....31

References....33

Appendix A: Important Source Code....35

Appendix B: Installation and Execution Instructions....37

Appendix C: Additional Screenshots....38

Appendix D: Experimental Results....39

LIST OF FIGURES

Figure 4.1: System flowchart of the digit recognition pipeline
Figure 4.2: Level-0 data flow diagram of the recognition system
Figure 4.3: Architecture of the feed-forward neural network used in the project
Figure 5.1: Model training and accuracy-graph code executed in Google Colab
Figure 5.2: Predictive system code for custom handwritten digit images
Figure 6.1: Epoch-wise training and validation output captured in Google Colab
Figure 6.2: Model accuracy over epochs (actual training graph)
Figure 6.3: Training and validation loss curves plotted from the observed epoch data
Figure 6.4: Custom input image containing the handwritten digit 3
Figure 6.5: Prediction output of the system for the handwritten digit 3
Figure C.1: Google Colab notebook interface during model training
Figure C.2: Complete predictive system execution in Google Colab

LIST OF TABLES

Table 3.1: Summary of the MNIST dataset
Table 4.1: Input and output specifications
Table 4.2: Functional requirements
Table 4.3: Non-functional requirements
Table 4.4: Hardware requirements
Table 4.5: Software requirements
Table 6.1: Actual training configuration used in the executed notebook
Table 6.2: Epoch-wise training and validation results
Table 6.3: Final-epoch (epoch 10) result summary
Table D.1: Consolidated experimental results

LIST OF ABBREVIATIONS

AI -: Artificial Intelligence
ANN -: Artificial Neural Network
API -: Application Programming Interface
CNN -: Convolutional Neural Network
CPU -: Central Processing Unit
DFD -: Data Flow Diagram
GPU -: Graphics Processing Unit
ML -: Machine Learning
MNIST -: Modified National Institute of Standards and Technology (dataset)
MLP -: Multilayer Perceptron
NIST -: National Institute of Standards and Technology
OCR -: Optical Character Recognition
ReLU -: Rectified Linear Unit
RGB -: Red, Green, Blue (colour model)

CHAPTER 1: INTRODUCTION

1.1 Introduction to Artificial Intelligence

Artificial Intelligence (AI) is the branch of computer science that deals with the design of systems capable of performing tasks that traditionally require human intelligence, such as visual perception, speech recognition, language understanding, decision-making, and pattern recognition. The central idea of AI is to enable machines to learn from data and adapt their behaviour to new situations rather than following only fixed, pre-programmed rules.

Over the past two decades, AI has progressed from rule-based expert systems to data-driven learning systems that power search engines, medical diagnosis tools, autonomous vehicles, and intelligent assistants. A particularly successful sub-field of AI is machine learning, in which systems improve automatically by discovering patterns in large collections of examples.

1.2 Introduction to Machine Learning

Machine Learning (ML) is a sub-field of artificial intelligence in which computers learn patterns from data instead of being explicitly programmed with hand-crafted rules. A learning algorithm is presented with training examples and adjusts its internal parameters to make accurate predictions on new, previously unseen data. Machine learning approaches are commonly divided into supervised, unsupervised, and reinforcement learning.

This project uses supervised learning, in which the model is trained on labelled examples: each training example pairs an input (an image of a handwritten digit) with a label (the correct digit from 0 to 9). Once trained, the model can classify new handwritten digit images that it has never seen before.

1.3 Introduction to Neural Networks

Artificial Neural Networks (ANNs) are computational models loosely inspired by biological neurons. A neural network consists of interconnected neurons organised into layers: an input layer that receives the data, one or more hidden layers that transform it, and an output layer that produces the prediction. Each connection carries a weight, and each neuron computes a weighted sum of its inputs, adds a bias, and passes the result through a non-linear activation function.

During training, the network's prediction is compared with the true label using a loss function, and the error is propagated backwards through the network by the backpropagation

algorithm. An optimiser such as gradient descent or Adam then updates the weights in the direction that reduces the loss. Repeating this process over many examples and epochs allows the network to learn useful internal representations, and multi-layer networks can learn highly complex, non-linear mappings, which makes them well suited for image classification.

1.4 Introduction to Computer Vision

Computer vision is the field of AI concerned with enabling machines to interpret visual information from images and videos. Typical tasks include image classification, object detection, segmentation, and optical character recognition. In a digital image, visual information is represented numerically: a grayscale image is a two-dimensional array of pixel intensities ranging from 0 (black) to 255 (white).

In handwritten digit recognition, the computer vision problem is to assign a category label (0-9) to an image of a handwritten digit. Because handwriting styles vary greatly in stroke thickness, slant, size, and shape, hand-crafted rules perform poorly, whereas learning-based approaches automatically learn the visual features that distinguish one digit from another.

1.5 Handwritten Digit Recognition

Handwritten digit recognition is the task of automatically identifying digits written by hand. It is a classic pattern recognition problem, challenging because the same digit can be written in many different ways, differing in size, position, orientation, stroke width, and style. Despite this variability, neural networks can learn robust representations that generalise well to unseen handwriting.

Reliable digit recognition has considerable practical importance: it is used in sorting postal mail by handwritten PIN codes, processing bank cheques, digitising paper forms, and converting handwritten answer sheets into digital records, reducing manual labour and human error.

1.6 MNIST Dataset Overview

The MNIST (Modified National Institute of Standards and Technology) dataset is the most widely used benchmark dataset for handwritten digit recognition. It was created by Yann LeCun, Corinna Cortes, and Christopher Burges from the original NIST databases and has become the standard introductory dataset for machine learning and deep learning.

The dataset contains 70,000 grayscale images of handwritten digits, each of resolution 28 by 28 pixels (784 pixel values per image), divided into 60,000 training images and 10,000 test images, with ten output classes for the digits 0 through 9. Because the images are already size-normalised and centred, MNIST allows students to focus on model design and training rather than data preparation. A detailed description is provided in Chapter 3.

1.7 Problem Statement

Manual reading and entry of handwritten digits is slow, expensive, and prone to human error. The problem addressed in this project is therefore: given a grayscale image of a single handwritten digit, automatically classify it into one of the ten digit classes (0-9) using a neural network trained on the MNIST dataset, and demonstrate the model's practical use through a predictive system that accepts custom handwritten digit images supplied by a user.

1.8 Aim of the Project

The aim of this project is to design, implement, train, and evaluate a neural network-based system for the automatic classification of handwritten digits using the MNIST dataset, and to demonstrate the trained model through an interactive predictive system capable of recognising custom handwritten digit images.

1.9 Objectives

- To study the fundamental concepts of artificial intelligence, machine learning, neural networks, and computer vision relevant to image classification.
- To study and understand the structure of the MNIST handwritten digit dataset.
- To preprocess image data through normalisation and reshaping so that it is suitable for training a neural network.
- To design and implement a feed-forward neural network using TensorFlow and Keras in Python.
- To train the model on the MNIST training set while monitoring training and validation accuracy and loss.
- To analyse the training behaviour of the model using accuracy and loss curves.
- To implement a predictive system that accepts a custom handwritten digit image, preprocesses it using OpenCV, and predicts the digit using the trained model.

- To evaluate the practical performance of the system and identify its advantages, limitations, and possible improvements.

1.10 Scope of the Project

The scope of this project covers the complete pipeline of a supervised image classification system: loading and preprocessing the MNIST dataset, constructing and training a feed-forward neural network, monitoring its performance, and deploying the trained model within a predictive system that classifies user-supplied handwritten digit images. The project is implemented in Python on Google Colab.

The project focuses on single-digit classification of 28 by 28 pixel grayscale images. Multi-digit sequence recognition, handwritten alphabet recognition, and real-time camera-based recognition are outside the present scope and are discussed as future work in Chapter 8.

1.11 Applications

- Postal automation: automatic sorting of mail by recognising handwritten postal codes.
- Banking: reading handwritten amounts and account numbers on cheques and deposit slips.
- Form processing: digitising numeric fields in paper forms, surveys, and application documents.
- Education: automatic evaluation of handwritten numeric responses in examinations.
- Data entry: converting handwritten numeric records into machine-readable databases.
- Assistive technology: helping visually impaired users interpret handwritten numbers.

1.12 Organisation of the Report

This report is organised into eight chapters. Chapter 1 introduces the background, problem statement, aim, objectives, scope, and applications. Chapter 2 reviews the literature on handwriting recognition and neural network approaches. Chapter 3 describes the methodology, the MNIST dataset, preprocessing, and the mathematical foundations of training. Chapter 4 presents the system design. Chapter 5 explains the complete implementation step by step. Chapter 6 presents and discusses the experimental results. Chapter 7 concludes the report, and Chapter 8 discusses future scope. References and appendices containing source code, installation instructions, additional screenshots, and consolidated results close the report.

CHAPTER 2: LITERATURE REVIEW

2.1 Development of Handwriting Recognition Systems

The automatic recognition of handwritten characters has been an active research area since the 1960s. Early OCR systems were designed mainly for printed text with fixed shapes and fonts; handwriting recognition proved far more difficult because of the enormous natural variation in human writing. Early approaches relied on template matching against stored prototypes and on structural analysis that decomposed characters into primitives such as lines, loops, and junctions.

A major step forward came with the application of statistical pattern recognition and, later, neural networks. The landmark work of LeCun and colleagues in the late 1980s and 1990s demonstrated that gradient-based learning applied to multi-layer networks could recognise handwritten ZIP codes with high accuracy, leading to systems deployed in postal automation and bank cheque reading [1].

2.2 Traditional OCR Techniques

Traditional OCR pipelines consist of several hand-engineered stages: binarisation, noise removal, skew correction, character segmentation, feature extraction, and classification. Common hand-crafted features include pixel density histograms, zoning features, projection profiles, and moment invariants, classified by methods such as k-nearest neighbours, decision trees, or support vector machines.

While these techniques performed reasonably on constrained problems, they share an important limitation: the features are designed by human experts and may not capture the most discriminative properties of the data. Performance degrades when writing style or image quality differs from the assumptions built into the features, which motivated the shift towards methods that discover features directly from data.

2.3 Machine Learning Approaches

With labelled datasets such as MNIST, machine learning classifiers could be trained directly on image data. Classical classifiers applied to MNIST include k-nearest neighbours, support vector machines, random forests, and boosted decision stumps, achieving error rates of roughly 1 to 5 percent; they are outperformed by neural networks, which jointly learn feature representations and classification boundaries.

The key advantage of neural network approaches is representation learning: the raw pixel values are provided to the network, and the hidden layers learn increasingly abstract features, from simple strokes in the early layers to complete digit shapes in the later layers.

2.4 Artificial Neural Networks

The theoretical foundation of neural networks dates back to the perceptron introduced by Rosenblatt in 1958 [12], a single-layer model limited to linearly separable patterns. The popularisation of the backpropagation algorithm by Rumelhart, Hinton, and Williams in 1986 [4] made it possible to train multi-layer networks efficiently by propagating error gradients backwards through the layers, reviving interest in connectionist models.

A feed-forward network with at least one hidden layer and non-linear activations is a universal function approximator: given enough neurons, it can approximate any continuous mapping from inputs to outputs, which makes such networks powerful tools for classification tasks such as digit recognition.

2.5 Multilayer Perceptrons

The Multilayer Perceptron (MLP) is the classical form of a feed-forward artificial neural network, consisting of an input layer, one or more fully connected hidden layers, and an output layer, with every neuron connected to all neurons in the adjacent layers. MLPs trained with backpropagation were among the first models to achieve strong results on handwritten digit recognition, and well-tuned MLPs reach accuracies of around 97 to 98 percent on MNIST.

The model implemented in this project is a feed-forward fully connected neural network of this type: the 28 by 28 input image is flattened into a vector of 784 values, processed by one or more dense hidden layers with non-linear activation, and classified by a ten-neuron softmax output layer.

2.6 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) extend the basic neural network architecture with convolutional and pooling layers that exploit the two-dimensional spatial structure of images. Convolutional layers apply learnable filters that detect local patterns such as edges and curves, while pooling layers reduce spatial dimensions and provide tolerance to small shifts and distortions. LeNet-5, developed by LeCun and colleagues for handwritten digit recognition, was one of the earliest successful CNNs [1].

Modern CNN variants have pushed MNIST accuracy above 99.5 percent; Ciresan, Meier, and Schmidhuber showed that committees of deep multi-column CNNs trained with data augmentation approach human-level performance on this benchmark [13]. CNNs are the natural architectural upgrade path for this project; the present work deliberately begins with a simpler feed-forward network, and CNN-based improvement is discussed as future scope in Chapter 8.

2.7 MNIST as a Benchmark Dataset

The MNIST dataset was assembled by LeCun, Cortes, and Burges from the handwritten digit databases of the U.S. National Institute of Standards and Technology [1], [2]. The images were size-normalised to fit a 20 by 20 pixel box, centred in a 28 by 28 pixel field by their centre of mass, and stored as grayscale intensities, with a standard split of 60,000 training and 10,000 test images.

MNIST became the de facto benchmark for evaluating classification algorithms because it is freely available, well-documented, balanced across the ten digit classes, and small enough to train on ordinary hardware while still posing a non-trivial recognition problem. Thousands of research papers report results on MNIST, making it possible to compare new methods against a large body of published work.

2.8 Previous Research and Related Work

Reported error rates on MNIST span more than two decades of progress. Early linear classifiers achieved roughly 8 to 12 percent error; support vector machines reduced it to about 1.4 percent; LeNet-5 and its variants brought it below 1 percent [1]; and deep CNNs with augmentation and ensembles reached approximately 0.2 to 0.3 percent [13]. Feed-forward networks with two hidden layers typically achieve error rates of about 1.5 to 2.5 percent, i.e. accuracies of roughly 97.5 to 98.5 percent.

These published results provide a useful frame of reference for the present project: the validation accuracy of approximately 97.4 percent obtained by the implemented feed-forward network is consistent with the performance range reported in the literature for simple dense architectures on MNIST.

2.9 Research Motivation

The literature shows a clear progression from hand-crafted feature engineering towards end-to-end learning with neural networks. For a student project in artificial intelligence,

handwritten digit recognition on MNIST is an ideal problem: well-defined, practically meaningful, computationally affordable, and supported by a rich body of prior work. The motivation of this project is to gain hands-on experience with the complete machine learning workflow, from dataset preparation and model design to training, evaluation, and deployment in a predictive system.

CHAPTER 3: METHODOLOGY

3.1 Research Methodology

This project follows an experimental research methodology consisting of six stages: (1) study of the problem domain and relevant literature; (2) acquisition and exploration of the MNIST dataset; (3) preprocessing of the image data; (4) design and implementation of a neural network classifier; (5) training of the model with systematic monitoring of accuracy and loss; and (6) evaluation of the trained model, including demonstration through a predictive system on custom handwritten images. Each stage builds on the results of the previous stage, and the experimental findings are recorded and analysed using quantitative measures (accuracy and loss) and visual evidence (training curves and prediction screenshots).

3.2 Dataset Collection

The MNIST dataset is publicly available and is bundled with the Keras deep learning library. It was loaded directly in the Google Colab notebook using the Keras datasets API, which automatically downloads the dataset and returns it as NumPy arrays. No manual data collection was required, and no personally identifiable information is involved, since the dataset consists of anonymised handwritten digit images collected from Census Bureau employees and high-school students by NIST [2].

3.3 Dataset Description

The dataset contains 70,000 grayscale images of handwritten digits. Table 3.1 summarises its key characteristics.

Table 3.1: Summary of the MNIST dataset

Property	Description
Total images	70,000
Training images	60,000
Test images	10,000
Image type	Grayscale (single channel)
Image dimensions	28 x 28 pixels (784 pixels per image)
Pixel value range	0 (black) to 255 (white)
Number of classes	10 (digits 0 to 9)

Property	Description
Label format	Integer class labels 0 to 9

The images are size-normalised and centred, and the ten classes are approximately balanced, with each digit represented by roughly 6,000 training images and 1,000 test images. These properties make MNIST well suited for supervised classification experiments.

3.4 Data Exploration

Before training, basic properties of the dataset were examined: the shapes of the training and test arrays were printed to confirm the expected dimensions (60,000 x 28 x 28 for the training images, 60,000 labels for the training set, and correspondingly 10,000 samples for the test set). Individual images were visualised with Matplotlib to confirm that the pixel intensities encode white or bright strokes on a dark background, and that each image is accompanied by the correct integer label. This sanity check ensures that the data pipeline feeds correctly formatted samples to the model.

3.5 Image Preprocessing

Preprocessing converts raw image data into a numerical form suitable for a neural network. Two distinct preprocessing pipelines are used in this project, and it is important to distinguish between them.

Training-time preprocessing. The MNIST images are already grayscale and of the correct size (28 x 28 pixels), so the only preprocessing required before training is pixel normalisation: every pixel intensity is divided by 255 so that the input values lie in the range 0 to 1. The same normalisation is applied to the test images.

Prediction-time preprocessing. A custom image supplied by the user is an ordinary colour or grayscale photograph of arbitrary size. Before it can be classified, it must be converted into the same format as the training data: the image is read with OpenCV, converted to grayscale, resized to 28 x 28 pixels, normalised by dividing by 255, and reshaped into the batch format expected by the model. Section 5.11 describes this pipeline in detail.

3.6 Grayscale Conversion

For custom input images, OpenCV converts the loaded image to a single-channel grayscale image using its colour conversion function. Grayscale conversion removes colour information and retains only intensity, which matches the format of the MNIST training

images and reduces the input dimensionality of the problem from three colour channels to one.

3.7 Image Resizing

Custom images are resized to 28 x 28 pixels using OpenCV's resize function so that their spatial dimensions exactly match the MNIST format. Resizing maps the handwritten digit onto the small fixed grid used during training, at the cost of some loss of fine detail, which is acceptable because the network was trained on images of the same resolution.

3.8 Normalisation

Pixel intensities in the raw images range from 0 to 255. Normalisation scales these values into the range 0 to 1 by dividing every pixel by 255:

$$x' = \frac{x}{255}$$

where x is the original pixel intensity and x' is the normalised value. Normalisation is important because gradient-based optimisation behaves better when input features have small, uniform scales: it keeps weight updates well conditioned and speeds up convergence during training.

3.9 Reshaping of Image Arrays

During training, the Keras model receives batches of images of shape (N, 28, 28), where N is the number of images in the batch; a Flatten layer inside the model converts each 28 x 28 image into a vector of 784 values. For prediction on a single custom image, the preprocessed image is reshaped from (28, 28) into (1, 28, 28) so that it forms a batch of one image, matching the input format the model was trained on.

3.10 Data Splitting

The MNIST dataset already provides a standard split of 60,000 training images and 10,000 test images. In addition, the training code executed in this project reserves a fraction of the training data for validation by passing the argument validation_split = 0.1 to the model.fit() function. This means that 10 percent of the training data (6,000 images) is held out during training and used after every epoch to compute validation accuracy and validation loss, giving an unbiased estimate of how well the model generalises to data it has not been trained on.

It should be noted that the project plan originally considered a larger validation fraction; however, the executed notebook uses a 10 percent validation split, and all results reported in Chapter 6 correspond to this actual executed configuration.

3.11 Neural Network Training

Training a neural network means adjusting its weights and biases so that the loss function is minimised. Each training pass consists of forward propagation, loss computation, backpropagation, and a weight update.

Forward propagation. For a neuron j in a layer, the weighted input is computed as z = sum over i of (w_i * x_i) + b, where w_i are the connection weights, x_i are the activations from the previous layer, and b is the bias. The neuron's output is a = f(z), where f is a non-linear activation function such as the Rectified Linear Unit (ReLU), defined as f(z) = max(0, z). ReLU is used in the hidden layers because it is computationally simple and avoids the vanishing gradient problem.

Output layer and softmax. The output layer contains ten neurons, one per digit class, and uses the softmax activation function, which converts the raw scores (logits) into a probability distribution over the ten classes:

$$softmax(z_k) = \frac{\exp(z_k)}{\exp(z_1) + \exp(z_2) + \ldots + \exp(z_{10})}$$

where z_k is the raw output (logit) of neuron k. The predicted class is the digit with the highest softmax probability, obtained with the argmax function.

3.12 Loss Function

The model is trained with the sparse categorical cross-entropy loss, which is the standard loss for multi-class classification with integer labels. For a single training example, the loss is the negative logarithm of the probability assigned to the true class:

$$L = -\log(p_y)$$

where p_y is the softmax probability that the model assigns to the correct class y. The loss is zero when the model is perfectly confident and correct, and grows large when the model assigns a low probability to the correct class. Averaging this loss over all examples in a batch gives the quantity that the optimiser minimises.

3.13 Optimisation

The network's weights are updated with the Adam optimiser, a widely used stochastic optimisation algorithm that adapts the learning rate of each parameter using estimates of the first and second moments of the gradients [5]. In its simplest form, gradient-based learning updates each parameter theta in the direction that decreases the loss:

$$theta_{new} = theta_{old} - lr * dL/d(theta)$$

where lr is the learning rate and dL/d(theta) is the gradient of the loss with respect to the parameter, computed by backpropagation. Adam refines this update by maintaining running averages of past gradients, which makes training faster and more stable than plain stochastic gradient descent.

3.14 Model Evaluation

Model performance is monitored using two metrics computed by Keras after every epoch: accuracy, the fraction of correctly classified images, and loss, the value of the cross-entropy loss function. Both metrics are computed on the training data and on the held-out validation data, producing the four quantities reported after each epoch: accuracy, loss, val_accuracy, and val_loss. Comparing training and validation curves reveals underfitting (both accuracies low), good fit (both high and close together), or overfitting (training accuracy high but validation accuracy stagnant or declining). The epoch-wise results and their analysis are presented in Chapter 6.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

The system follows a layered architecture of four logical components: (1) a data layer that loads the MNIST dataset and reads custom user images; (2) a preprocessing layer that normalises, resizes, converts, and reshapes images; (3) a model layer containing the neural network; and (4) a presentation layer in the Colab notebook that displays training progress, graphs, and predictions. This separation keeps each stage independently testable.

4.2 Overall Workflow

The overall workflow of the system, illustrated in Figure 4.1, proceeds as follows. The MNIST dataset is loaded and normalised. The neural network is constructed, compiled with the Adam optimiser and the sparse categorical cross-entropy loss, and trained for ten epochs with a 10 percent validation split, while accuracy and loss are recorded for the training and validation data. After training, the history is used to plot the accuracy curves. The trained model is then used by the predictive system: a user-supplied image is preprocessed with OpenCV and classified, and the predicted digit is displayed.

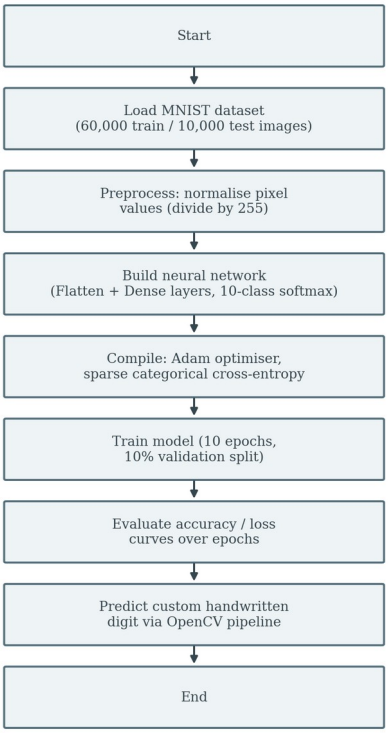


Figure 4.1: System flowchart of the digit recognition pipeline

4.3 Input and Output Specifications

Table 4.1: Input and output specifications

Item	Specification	Description
Training input	MNIST arrays (60,000 x 28 x 28)	Grayscale digit images with integer labels 0-9
Prediction input	Image file path	User-supplied image of a handwritten digit
Model output	Probability vector (10 values)	Softmax probability for each digit class
System output	Predicted digit (0-9)	Class with the highest prediction score

4.4 Functional Requirements

Table 4.2: Functional requirements

ID	Requirement
FR1	The system shall load the MNIST dataset and split it into training, validation, and test data.
FR2	The system shall normalise pixel values to the range 0 to 1 before training.
FR3	The system shall train a neural network and record accuracy and loss after every epoch.
FR4	The system shall plot the training and validation accuracy curves.
FR5	The system shall accept a custom image path from the user.
FR6	The system shall preprocess the custom image (grayscale, resize, normalise, reshape).
FR7	The system shall predict and display the digit contained in the custom image.

4.5 Non-Functional Requirements

Table 4.3: Non-functional requirements

ID	Requirement
NFR1	Performance: training of ten epochs shall complete within a few minutes on Colab CPU.
NFR2	Accuracy: the model shall achieve validation accuracy above 90 percent.
NFR3	Usability: the predictive system shall require only an image path as input.

ID	Requirement
NFR4	Portability: the system shall run in a standard Google Colab environment.
NFR5	Reliability: preprocessing shall handle user images of arbitrary size and colour format.
NFR6	Maintainability: the code shall be organised into clear, commented steps.

4.6 Hardware Requirements

Table 4.4: Hardware requirements

Component	Requirement
Processor	Standard multi-core CPU (Google Colab runtime)
Memory (RAM)	4 GB or higher (approx. 12 GB provided by Colab)
Storage	2 GB free disk space for dataset and environment
GPU	Optional; not required for the implemented model
Display	Any display capable of running a web browser

4.7 Software Requirements

Table 4.5: Software requirements

Software	Purpose
Python 3	Programming language of the implementation
Google Colab	Cloud-based Jupyter notebook environment
TensorFlow / Keras	Neural network construction, training, and prediction
NumPy	Numerical array operations and reshaping
OpenCV (cv2)	Custom image reading, grayscale conversion, and resizing
Matplotlib	Plotting of training and validation accuracy graphs

4.8 Data Flow Diagram

Figure 4.2 shows the level-0 data flow diagram of the system. The user provides a handwritten digit image as input; the system preprocesses the image, passes it to the trained neural network, and returns the predicted digit class. During training, the external input is the MNIST dataset, and the output is the trained model with its performance curves.

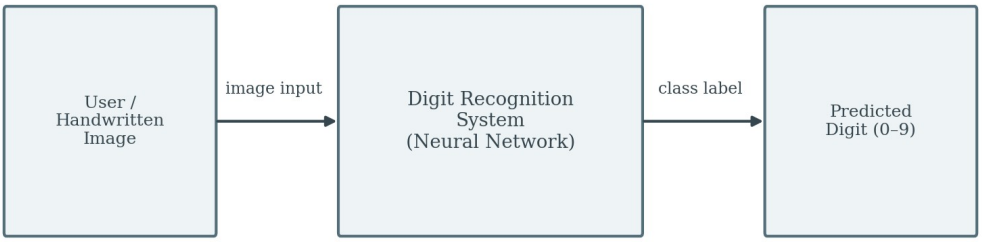


Figure 4.2: Level-0 data flow diagram of the recognition system

4.9 System Flowchart

The system flowchart has already been presented in Figure 4.1. It traces the sequential flow from dataset loading through preprocessing, model construction, compilation, training, and evaluation to the final prediction stage.

4.10 Neural Network Architecture

The executed notebook implements a feed-forward (fully connected) neural network using the Keras Sequential API, illustrated in Figure 4.3. The input layer receives a 28 x 28 grayscale image and flattens it into a vector of 784 values; this is confirmed by the prediction code, which reshapes each image to (1, 28, 28) without any convolutional channel dimension, indicating a dense (MLP-style) architecture. The flattened input passes through dense hidden layers with ReLU activation, and the output layer contains ten softmax neurons producing a probability distribution over the ten digit classes; the predicted class is the argmax of these probabilities. The network is compiled with the Adam optimiser and the sparse categorical cross-entropy loss.

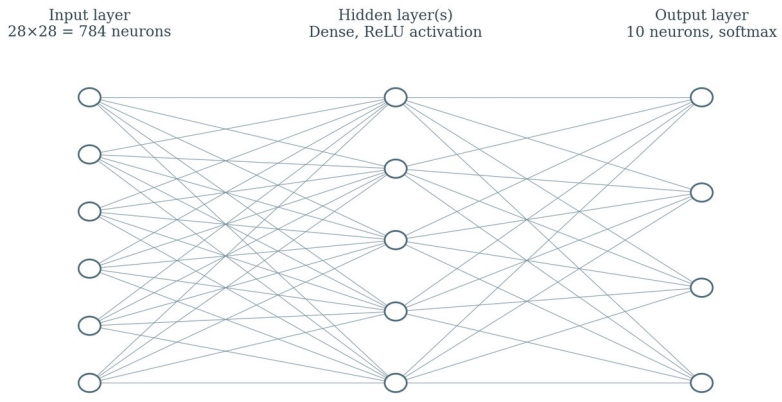


Figure 4.3: Architecture of the feed-forward neural network used in the project

CHAPTER 5: IMPLEMENTATION

5.1 Google Colab Environment Setup

The project was implemented in Google Colab, a free cloud-based Jupyter notebook environment that provides a preconfigured Python runtime with the major machine learning libraries already installed. Using Colab removes the need for local installation and provides sufficient CPU and memory resources for training the model. The notebook, named MNIST Digit.ipynb, was executed cell by cell, and the outputs, including the epoch-wise training logs and the accuracy graph, were captured as the experimental evidence presented in Chapter 6 and the appendices.

5.2 Library Installation and Importing

The implementation begins by importing the required Python libraries. TensorFlow provides the Keras API used to build and train the neural network; NumPy provides array operations; Matplotlib is used to plot the accuracy curves; and OpenCV (cv2) together with the Colab helper cv2_imshow is used to read, display, and preprocess custom images.

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
from google.colab.patches import cv2_imshow
import tensorflow as tf
from tensorflow import keras
```

5.3 Dataset Loading

The MNIST dataset is loaded through the Keras datasets API, which downloads the data on first use and returns four NumPy arrays: the training images and labels, and the test images and labels.

```
# loading the MNIST dataset from Keras
mnist = keras.datasets.mnist
(X_train, Y_train), (X_test, Y_test) = mnist.load_data()
```

After loading, X_train has shape (60000, 28, 28), Y_train has shape (60000,), X_test has shape (10000, 28, 28), and Y_test has shape (10000,), matching the dataset description in Chapter 3.

5.4 Dataset Preprocessing

The pixel values of the training and test images are normalised by dividing by 255, scaling them from the range 0-255 to the range 0-1. The labels are integer class values from 0

to 9, which is exactly the format expected by the sparse categorical cross-entropy loss, so no one-hot encoding of the labels is required.

```
# normalising the pixel values to the range 0 to 1
X_train = X_train / 255
X_test = X_test / 255
```

5.5 Model Construction

The neural network is constructed using the Keras Sequential API as a stack of layers. The first layer is a Flatten layer that converts each 28 x 28 image into a vector of 784 values. It is followed by one or more fully connected (Dense) hidden layers with the ReLU activation function, and a final Dense output layer with ten neurons and softmax activation, one neuron per digit class. The architecture is a feed-forward multilayer perceptron rather than a convolutional network, which is consistent with the input shape (1, 28, 28) used by the prediction code.

```
# building the feed-forward neural network
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(units=..., activation='relu'),   # hidden layer(s)
    keras.layers.Dense(units=10, activation='softmax')  # output layer
])
```

5.6 Model Compilation

Compilation configures the learning process. The model is compiled with the Adam optimiser, the sparse categorical cross-entropy loss function, and accuracy as the monitored metric.

```
# compiling the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

5.7 Model Training

The model is trained with the model.fit() function on the training images and labels. As shown in the executed notebook (Figure 5.1), the training run used 10 epochs and a validation split of 0.1, meaning that 10 percent of the training data was reserved for computing validation accuracy and loss after each epoch. The function returns a history object that records all four metrics after every epoch.

```
# training the Neural Network and capturing history
history = model.fit(X_train, Y_train, epochs=10, validation_split=0.1)
```



```
# training the Neural Network and capturing history
history = model.fit(X_train, Y_train, epochs=10, validation_split=0.1)

# plotting the training accuracy graph
plt.figure(figsize=(8, 5))
plt.plot(history.history['accuracy'], label='Training Accuracy', color='blue', marker='o')
if 'val_accuracy' in history.history:
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy', color='orange', marker='x')

# Adding a line at 90% accuracy
plt.axhline(y=0.90, color='red', linestyle='--', label='90% Accuracy Threshold')

plt.title('Model Accuracy over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.ylim(0.85, 1.0) # Zooming in on the 85%-100% range to clearly see accuracy above 90%
plt.legend()
plt.grid(True)
plt.show()
```

Figure 5.1: Model training and accuracy-graph code executed in Google Colab

5.8 Training History

The history object returned by model.fit() stores the training accuracy, training loss, validation accuracy, and validation loss for each epoch in its history dictionary. These values are used both for plotting the accuracy graph and for the quantitative analysis presented in Chapter 6. The epoch-wise log printed during training is shown in Figure 6.1.

5.9 Accuracy and Loss Graph Generation

After training, Matplotlib is used to plot the training accuracy and the validation accuracy over the ten epochs. A horizontal dashed line marks the 90 percent accuracy threshold for reference, and the vertical axis is zoomed to the 0.85-1.00 range so that the behaviour above 90 percent is clearly visible. The plotting code is included in Figure 5.1 and the resulting graph is presented in Figure 6.2.

```
# plotting the training accuracy graph
plt.figure(figsize=(8, 5))
plt.plot(history.history['accuracy'], label='Training Accuracy',
         color='blue', marker='o')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy',
         color='orange', marker='x')
plt.axhline(y=0.90, color='red', linestyle='--',
            label='90% Accuracy Threshold')
plt.title('Model Accuracy over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.ylim(0.85, 1.0)
plt.legend()
plt.grid(True)
plt.show()
```

5.10 Custom Image Input

The predictive system, shown in Figure 5.2, begins by asking the user to enter the path of the image to be predicted. In the demonstrated run, the user provided the file 3-digit.PNG, which contains a handwritten digit 3.

5.11 Image Preprocessing Using OpenCV

The custom image passes through the prediction-time preprocessing pipeline described in Chapter 3: OpenCV reads the image from the given path; the image is displayed with cv2_imshow; it is converted to grayscale; it is resized to 28 x 28 pixels; the pixel values are normalised by dividing by 255; and the image is reshaped to (1, 28, 28), producing a batch of one image in exactly the format the model was trained on.

5.12 Model Prediction

The trained model's predict() function is applied to the preprocessed image, returning a vector of ten softmax probabilities, one per digit class. NumPy's argmax function then selects the index of the highest probability, which is the predicted digit label.

5.13 Output Generation

Finally, the system prints the recognised digit. The complete predictive system code, exactly as executed in the notebook, is shown in Figure 5.2.

```
input_image_path = input('Path of the image to be predicted: ')
input_image = cv2.imread(input_image_path)
cv2_imshow(input_image)
grayscale = cv2.cvtColor(input_image, cv2.COLOR_RGB2GRAY)
input_image_resize = cv2.resize(grayscale, (28, 28))
input_image_resize = input_image_resize / 255
image_reshaped = np.reshape(input_image_resize, [1, 28, 28])
input_prediction = model.predict(image_reshaped)
input_pred_label = np.argmax(input_prediction)
print('The Handwritten Digit is recognised as ', input_pred_label)
```

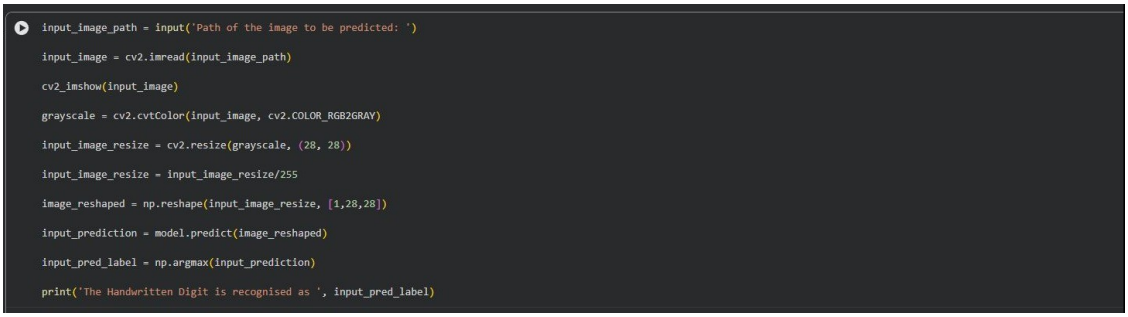


Figure 5.2: Predictive system code for custom handwritten digit images

The demonstrated execution of this code, including the displayed input image and the printed prediction, is presented and discussed in Section 6.8.

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Experimental Setup

All experiments were conducted in Google Colab using a standard CPU runtime. The dataset, preprocessing, model, and training procedure were exactly those described in Chapters 3 and 5. The results reported in this chapter were captured directly from the executed notebook and represent the actual observed behaviour of the implemented system.

6.2 Actual Training Configuration

Table 6.1 summarises the training configuration used in the executed notebook, as evidenced by the training code and log shown in Figures 5.1 and 6.1. The executed configuration uses a 10 percent validation split; this is the configuration to which all reported results correspond.

Table 6.1: Actual training configuration used in the executed notebook

Parameter	Value
Framework	TensorFlow / Keras (Python, Google Colab)
Training function	model.fit()
Number of epochs	10
Validation split	0.1 (10% of the training data)
Optimiser	Adam
Loss function	Sparse categorical cross-entropy
Monitored metrics	accuracy, loss, val_accuracy, val_loss
Monitored per epoch	1688/1688 training steps

6.3 Epoch-Wise Training Results

Figure 6.1 shows the epoch-wise training output captured from the notebook, and Table 6.2 reproduces the observed values in tabular form. Each epoch processed 1688 training steps.



Figure 6.1: Epoch-wise training and validation output captured in Google Colab

Table 6.2: Epoch-wise training and validation results

Epoch	Training Accuracy	Training Loss	Validation Accuracy	Validation Loss
1	90.32%	0.3290	95.33%	0.1559
2	95.51%	0.1512	96.65%	0.1152
3	96.73%	0.1097	97.05%	0.1042
4	97.37%	0.0872	97.40%	0.1005
5	97.93%	0.0713	97.35%	0.0994
6	98.25%	0.0592	97.40%	0.1041
7	98.53%	0.0502	97.27%	0.1097
8	98.67%	0.0439	97.22%	0.1178
9	98.89%	0.0376	97.00%	0.1246
10	98.93%	0.0342	97.40%	0.1222

It is emphasised that these figures are training and validation results recorded during model fitting; they are not official test-set accuracy values, and no test-set evaluation, confusion matrix, precision, recall, or F1-score is claimed in this report.

6.4 Training and Validation Accuracy

Figure 6.2 shows the accuracy graph produced by the notebook. Training accuracy rises steeply during the first three epochs, from 90.32 percent to 96.73 percent, and then increases more gradually, reaching 98.93 percent at epoch 10. Validation accuracy starts higher (95.33 percent at epoch 1, being measured after the epoch's weight updates), improves to about 97.4 percent by epoch 4, and then fluctuates slightly between 97.0 and 97.4 percent. The notebook printed a final training data accuracy of 98.9 percent, consistent with the epoch-10 value.

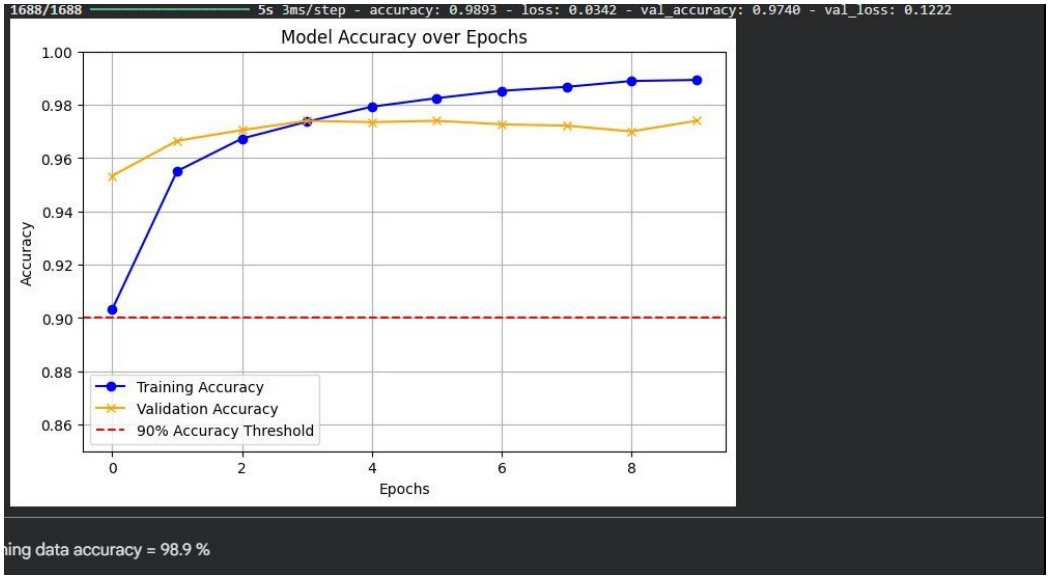


Figure 6.2: Model accuracy over epochs (actual training graph)

6.5 Training and Validation Loss

Figure 6.3 plots the loss values from Table 6.2. Training loss decreases smoothly and monotonically from 0.3290 to 0.0342 across the ten epochs, showing that the optimiser consistently reduces the error on the training data. Validation loss decreases from 0.1559 to a minimum of 0.0994 at epoch 5, after which it begins to fluctuate and rise slightly, ending at 0.1222. The divergence between the steadily falling training loss and the slowly rising validation loss after about epoch 5 is a classic early sign of overfitting.

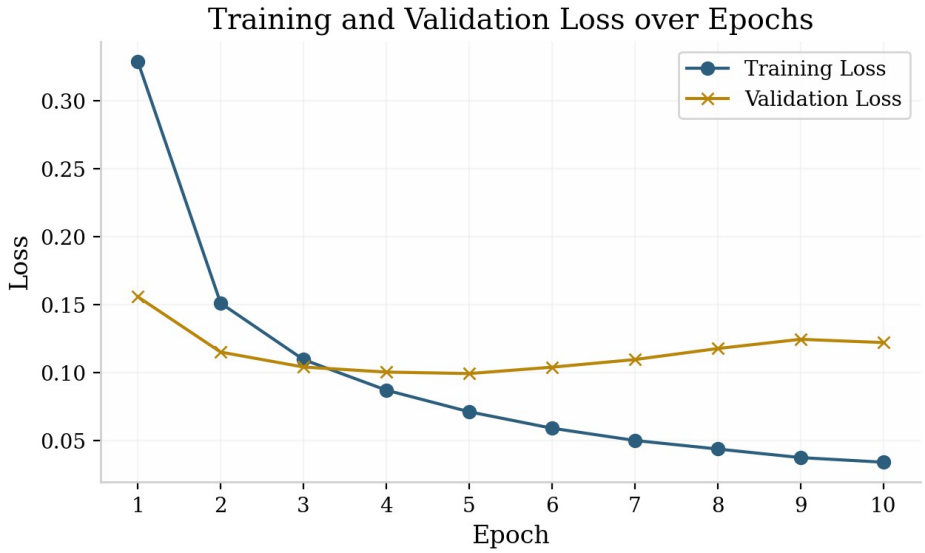


Figure 6.3: Training and validation loss curves plotted from the observed epoch data

6.6 Discussion of Model Performance

The results demonstrate that the feed-forward neural network learned the digit classification task effectively. Three observations stand out. First, the model converged

quickly: over 95 percent training accuracy was reached within two epochs, reflecting both the simplicity of the MNIST task and the effectiveness of normalised inputs with the Adam optimiser. Second, the validation accuracy of approximately 97.4 percent shows good generalisation to held-out data and is consistent with published results for simple dense networks on MNIST (typically 97 to 98 percent). Third, the growing gap between training and validation performance in the later epochs indicates mild overfitting: by epoch 10 the model fits the training data better (98.93 percent) than the validation data (97.40 percent), and the validation loss has risen from its epoch-5 minimum. In practice, training could have been stopped around epochs 4 to 6 with almost no loss of validation performance, or regularisation techniques such as dropout could be introduced to close the gap.

6.7 Final-Epoch Summary

Table 6.3: Final-epoch (epoch 10) result summary

Parameter	Actual Observed Value
Total epochs	10
Final training accuracy	98.93%
Final training loss	0.0342
Final validation accuracy	97.40%
Final validation loss	0.1222

6.8 Predictive System Demonstration

To demonstrate the practical use of the trained model, the predictive system described in Chapter 5 was executed with a custom handwritten digit image. The user supplied the file 3-digit.PNG, shown in Figure 6.4, which contains a handwritten digit 3 in white strokes on a dark background, matching the MNIST visual format.



Figure 6.4: Custom input image containing the handwritten digit 3

The system read the image with OpenCV, displayed it, converted it to grayscale, resized it to 28 x 28 pixels, normalised the pixel values, reshaped the array to (1, 28, 28), and passed it to the trained model. As shown in Figure 6.5, the system printed the message: The Handwritten Digit is recognised as 3. The prediction is correct: the model successfully recognised the user's handwritten digit.



Figure 6.5: Prediction output of the system for the handwritten digit 3

This demonstration confirms that the preprocessing pipeline correctly converts arbitrary user images into the MNIST format and that the trained model transfers successfully from the training data to fresh handwritten input. The same procedure can be applied to any handwritten digit image supplied by a user, which makes the system a practical foundation for applications such as form processing and digitisation of handwritten numeric entries.

6.9 Advantages of the System

- High accuracy: approximately 97.4 percent validation accuracy with a simple architecture.
- Fast training: ten epochs complete in about a minute on a free Colab CPU runtime.
- End-to-end pipeline: from dataset loading to user-facing prediction in a single notebook.

- Flexible input: the predictive system accepts custom images of arbitrary size and colour format.
- Reproducibility: the code, configuration, and results are fully documented in this report.
- Low resource requirements: no GPU or specialised hardware is needed.

6.10 Limitations of the System

- The model is a simple feed-forward network; convolutional architectures can reach still higher accuracy.
- Mild overfitting is visible in the later epochs, as discussed in Section 6.6.
- The system recognises one digit per image; it does not segment or read multi-digit numbers.
- Custom images must resemble the MNIST format (light digit on dark background) for best results.
- Performance is reported as training and validation metrics; a formal test-set evaluation is left to future work.
- Very slanted, noisy, or poorly written digits may be misclassified.

CHAPTER 7: CONCLUSION

7.1 Project Summary

This project designed, implemented, trained, and evaluated a neural network system for handwritten digit classification using the MNIST dataset, and demonstrated the trained model through an interactive predictive system. A feed-forward network was built with TensorFlow and Keras in Google Colab and trained for ten epochs with a 10 percent validation split.

7.2 Achievement of Objectives

All objectives stated in Chapter 1 were achieved: the dataset was studied, loaded, and preprocessed; the network was constructed, compiled, and trained; performance was monitored and visualised; and the predictive system classified custom images with OpenCV preprocessing. The system achieved a final training accuracy of 98.93 percent and a validation accuracy of 97.40 percent, comfortably exceeding the 90 percent target set in the design requirements.

7.3 Model Performance and Successful Prediction

The analysis in Chapter 6 shows rapid convergence, stable validation accuracy around 97 percent, and only mild overfitting towards the end of training, consistent with published results for dense architectures on MNIST. The demonstration, in which the system correctly recognised a user's handwritten digit 3, validates the complete pipeline from image acquisition to classification and confirms that the model generalises beyond its training data.

7.4 Learning Outcomes

The project provided hands-on experience with the complete machine learning workflow: dataset handling, preprocessing, network design, training, performance visualisation, and deployment of a trained model in a working predictive system. It also reinforced concepts such as forward propagation, activation functions, softmax classification, cross-entropy loss, backpropagation, and gradient-based optimisation.

7.5 Overall Conclusion

This project confirms that even a modest feed-forward architecture, trained with standard techniques on MNIST, delivers accurate and practically useful handwritten digit classification, and forms a solid foundation for the advanced capabilities outlined in Chapter 8.

CHAPTER 8: FUTURE SCOPE

8.1 Real-Time and Webcam-Based Digit Recognition

The current system classifies digit images provided as files. Natural extensions are real-time recognition, in which digits written on a digital canvas are classified instantly as the user draws them, and webcam-based recognition, in which OpenCV captures video frames and the preprocessing pipeline, extended with thresholding and contour detection, extracts the digit region and feeds it to the model continuously.

8.2 Mobile and Web Application Development

The trained model can be converted to a compact format such as TensorFlow Lite and embedded in a mobile application for on-device recognition. Likewise, deploying the model behind a web interface with a browser drawing canvas would turn the project into a complete end-user product.

8.3 Improved Neural Network Architecture

Replacing the feed-forward network with a Convolutional Neural Network would exploit the spatial structure of the images and is expected to raise accuracy beyond 99 percent. Further improvements include dropout regularisation against the mild overfitting observed here, batch normalisation, data augmentation, and early stopping based on validation loss.

8.4 Larger Datasets and Handwritten Alphabets

Training and evaluating on larger and more diverse datasets, such as EMNIST, would improve robustness to different writing styles. The same pipeline also extends naturally from digits to handwritten letters and full words; combined with segmentation techniques, this would move the project from single-character classification towards general handwritten text recognition.

8.5 Improved Accuracy, Robustness, and Deployment

Formal evaluation on the held-out test set, with precision, recall, F1-score, and a confusion matrix, would give a complete picture of per-class performance. Finally, the trained model can be saved, versioned, and deployed as a reusable prediction service, allowing applications such as form-processing or banking software to consume digit recognition through an API.

REFERENCES

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.

[2] Y. LeCun, C. Cortes, and C. J. C. Burges, "The MNIST database of handwritten digits," [Online]. Available: http://yann.lecun.com/exdb/mnist/

[3] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," Nature, vol. 323, no. 6088, pp. 533-536, 1986.

[5] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. 3rd Int. Conf. on Learning Representations (ICLR), San Diego, CA, USA, 2015.

[6] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous distributed systems," Google Research, 2016. [Online]. Available: https://www.tensorflow.org/

[7] F. Chollet et al., "Keras," 2015. [Online]. Available: https://keras.io/

[8] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, vol. 25, no. 11, pp. 120-123, 2000.

[9] A. Geron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.

[10] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.

[11] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, no. 7825, pp. 357-362, 2020.

[12] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," Psychological Review, vol. 65, no. 6, pp. 386-408, 1958.

[13] D. Ciresan, U. Meier, and J. Schmidhuber, "Multi-column deep neural networks for image classification," in Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), Providence, RI, USA, 2012, pp. 3642-3649.

APPENDIX A: IMPORTANT SOURCE CODE

A.1 Complete Training Pipeline

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
from google.colab.patches import cv2_imshow
import tensorflow as tf
from tensorflow import keras

# loading the MNIST dataset
mnist = keras.datasets.mnist
(X_train, Y_train), (X_test, Y_test) = mnist.load_data()

# normalising the pixel values
X_train = X_train / 255
X_test = X_test / 255

# building the feed-forward neural network
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(units=..., activation='relu'),   # hidden layer(s)
    keras.layers.Dense(units=10, activation='softmax')  # output layer
])

# compiling the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# training the Neural Network and capturing history
history = model.fit(X_train, Y_train, epochs=10, validation_split=0.1)
```

A.2 Accuracy Graph Plotting

```
plt.figure(figsize=(8, 5))
plt.plot(history.history['accuracy'], label='Training Accuracy',
         color='blue', marker='o')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy',
         color='orange', marker='x')
plt.axhline(y=0.90, color='red', linestyle='--',
            label='90% Accuracy Threshold')
plt.title('Model Accuracy over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.ylim(0.85, 1.0)
plt.legend()
plt.grid(True)
plt.show()
```

A.3 Predictive System

```
input_image_path = input('Path of the image to be predicted: ')
input_image = cv2.imread(input_image_path)
cv2_imshow(input_image)
```

```
grayscale = cv2.cvtColor(input_image, cv2.COLOR_RGB2GRAY)
input_image_resize = cv2.resize(grayscale, (28, 28))
input_image_resize = input_image_resize / 255
image_reshaped = np.reshape(input_image_resize, [1, 28, 28])
input_prediction = model.predict(image_reshaped)
input_pred_label = np.argmax(input_prediction)
print('The Handwritten Digit is recognised as ', input_pred_label)
```

APPENDIX B: INSTALLATION AND EXECUTION INSTRUCTIONS

B.1 Running in Google Colab (Recommended)

- Open https://colab.research.google.com in a web browser and sign in with a Google account.
- Create a new notebook and name it, for example, MNIST Digit.ipynb.
- Copy the code cells from Appendix A into the notebook in the given order.
- Select Runtime > Run all to execute the complete pipeline.
- For prediction, upload a handwritten digit image to the Colab file panel and provide its path (for example, /content/3-digit.PNG) when prompted.

B.2 Running Locally

- Install Python 3.9 or later.
- Install the required libraries: pip install tensorflow numpy opencv-python matplotlib
- Save the code as a .py file or .ipynb notebook and run it with Python or Jupyter.
- Note: cv2_imshow is specific to Colab; locally, use cv2.imshow('image', input_image) followed by cv2.waitKey(0).

APPENDIX C: ADDITIONAL SCREENSHOTS

Figures C.1 and C.2 present the complete screenshots of the executed Google Colab notebook from which the cropped figures in Chapters 5 and 6 were taken. They serve as the original, unedited evidence of the training process and the predictive system execution.

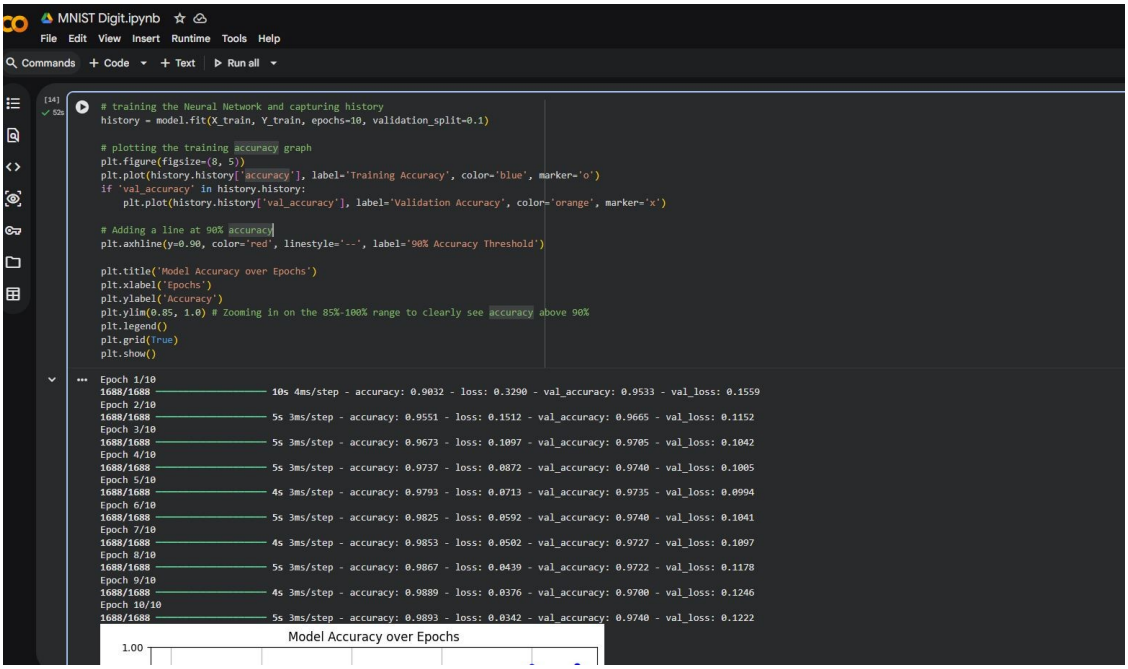


Figure C.1: Google Colab notebook interface during model training

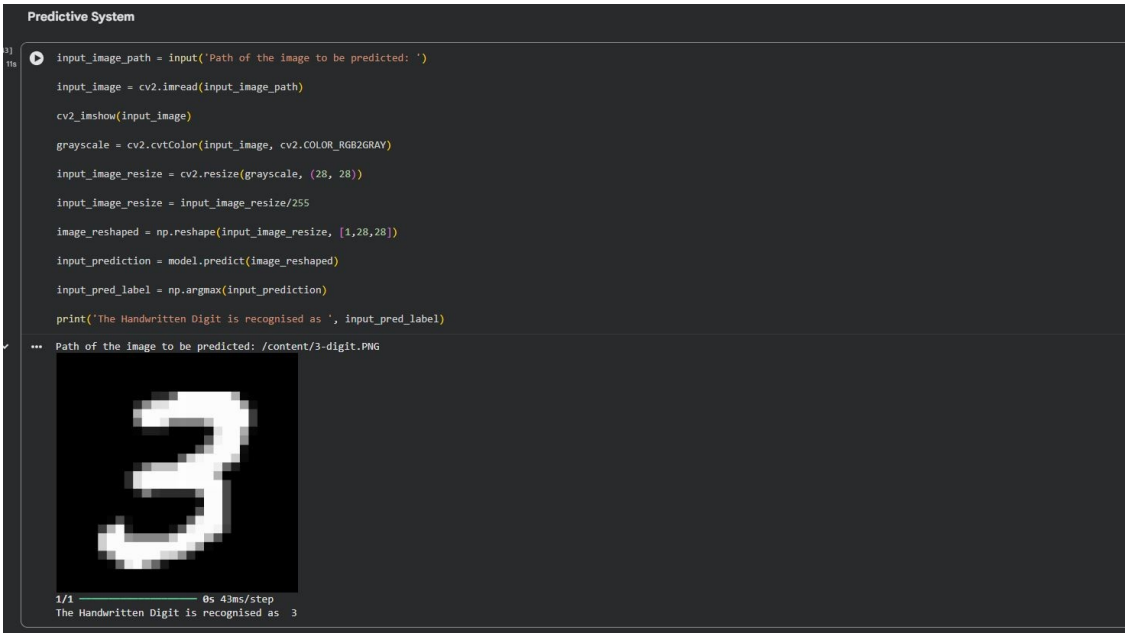


Figure C.2: Complete predictive system execution in Google Colab

APPENDIX D: EXPERIMENTAL RESULTS

The complete epoch-wise results are presented in Table 6.2 (Chapter 6); they were read directly from the epoch-wise log of the executed notebook (Figure 6.1). The consolidated summary of the experiment is given below.

Table D.1: Consolidated experimental results

Item	Observed Value
Total epochs	10
Training steps per epoch	1688
Final training accuracy	98.93%
Final training loss	0.0342
Final validation accuracy	97.40%
Final validation loss	0.1222
Best validation accuracy	97.40% (epochs 4, 6, and 10)
Minimum validation loss	0.0994 (epoch 5)
Prediction demonstration	Handwritten digit 3 correctly recognised as 3

All values in this report originate from the actual executed notebook; no test-set accuracy, precision, recall, F1-score, or confusion matrix values are claimed, as these were not part of the executed experiment.